CODE SPORT

Sandya Mannarswamy

This month's column includes a detailed discussion on a couple of coding questions as well as a few more interview questions.

We have been discussing many interview questions over the last couple of months. In this column, let us take up a couple of coding questions, discuss the approach to solving them, before covering a few interview questions.

A key point to remember when preparing for a coding interview is to practice each question as if you are actually giving the interview. Start with outlining an algorithm and then writing pseudo-code for it. Quantify the time and space complexity of your approach. Check whether this satisfies the conditions given in the question. For example, coding an obvious brute force O(N^2) solution may not fetch you any points with the interviewer even if you write perfect code. On the other hand, the pseudo-code for even a simple but sophisticated O(N) solution may get you through the interview. So do take the time to come up with a solution that is within the time and space complexity constraints specified in the problem's description.

Then list all the various test cases you can think of, including corner cases. Don't forget to consider boundary conditions. Once you are satisfied that your pseudo-code can handle these cases, write the code in your favourite programming language, run it with the test cases you had created earlier, and verify that it works correctly.

Let us move on to our first problem.

1. You are given two sorted arrays of integers – Array A of size N1 and Array B of size N2, sorted in ascending order and a positive integer K. You are asked to write code to identify the K smallest pairs of sums from the two arrays. A pair consists of one number from Array A and another number from Array B. For example, given that Array A = [1, 7, 11] and B = [2, 4, 6], K = 3, the correct output consists of [[1,2], [1,4], [1.6]].

Let us start off with the simplest approach. We can compute the sum of all pairs and then sort the pairs to find the first K smallest pairs. This will result in O(N1 * N2), which is O(n^2) where N1 = N2 = n. In this case, we are not utilising the fact that the two arrays are sorted. Since the arrays are sorted, we need to consider only the first K elements from each of the two arrays for computing the sums. We can get the K*K pair sums of these K elements. We can either sort these sums to find the first K smallest pairs, or we can build a heap of size K from these K^2 pair sums such that the heap contains the first K smallest pair sums. This will allow you to find a solution, the complexity of which is much lower than O(n^2).

What are the possible corner test cases? Any of the two arrays can be empty, or both can be. Both the arrays can contain thousands of integers. Make sure that your code scales for large-sized arrays. Also remember that since the arrays can contain any integers, you can handle integer overflow when summing up the two elements. So consider test cases where an array element is close to INT_MAX. Also, the array may contain duplicates. The value of K can be such that (N1 * N2) can be less than K. Make sure that your code contains all these different examples correctly. Consider the